

API Miam N Go

Production, Authentification, Sécurité par rôles

et Référentiel officiel des statuts métier

Backend Laravel REST - Version consolidée v1.3

Projet	Miam N Go
Version	1.3 - Production sécurisée + statuts métier
Domaine production	https://api-miamngo.digiapp.tech
Base API	https://api-miamngo.digiapp.tech/api
Repository GitHub	https://github.com/BabyBossKfy/miam_n_go
Framework	Laravel 12 + Sanctum
Date de mise à jour	16 juillet 2026
Auteur	Franck Kianguebeni

Statut : API déployée en production, sécurisée par tokens Sanctum, contrôlée par rôles ADMIN, PARTNER et CUSTOMER, avec référentiel officiel des statuts métier.

1. Résumé exécutif

Le backend Miam N Go est une API REST Laravel structurée autour du catalogue public, de l'authentification par token, de la gestion des commandes, du paiement, de la livraison et d'une sécurité par rôles.

Cette version v1.3 consolide la documentation v1.2 et ajoute le référentiel officiel des statuts métier : `status_order`, `status_payment` et `status_delivery`.

Flux métier couvert : Inscription client - Connexion - Consultation produits - Commande - OrderDetails - Paiement - Livraison - Suivi selon rôle.

2. Informations de production

Paramètre	Valeur
Domaine API	https://api-miamngo.digiapp.tech
URL de base	https://api-miamngo.digiapp.tech/api
Framework	Laravel 12
Sécurité API	Laravel Sanctum avec Bearer Token
Base de données	MySQL
Hébergement	DigiApp
Document Root	/public_html/api-miamngo/public

3. Nouvelles fonctionnalités ajoutées

Groupe	Fonctionnalités intégrées
Authentification	Inscription client, connexion, génération de token API, /auth/me, déconnexion.
Rôles	ADMIN, PARTNER, CUSTOMER avec droits différenciés.
Relations sécurité	users vers customers, users vers partners via partner_users.
Produits	PARTNER peut gérer uniquement les produits de son restaurant.
Commandes	CUSTOMER voit ses commandes, PARTNER voit les commandes contenant ses produits, ADMIN voit tout.
Clients	PARTNER voit uniquement les clients ayant commandé ses produits.
Paielements	Paieement Mobile Money, téléphone de paieement, paieement à la livraison.
Livraisons	Livraisons consultables ou modifiables selon le rôle.
Statuts métier	Référentiel officiel status_order, status_payment, status_delivery.

4. Architecture de sécurité par rôles

Rôle	Description	Droits principaux
ADMIN	Utilisateur administrateur global.	Voir et gérer toutes les ressources : clients, produits, commandes, paiements, livraisons.
PARTNER	Utilisateur lié à un ou plusieurs restaurants.	Voir ses produits, commandes contenant ses produits, clients associés, paiements et livraisons associés.
CUSTOMER	Client final de l'application.	S inscrire, se connecter, consulter le catalogue, créer ses commandes, payer ses commandes, voir ses livraisons.

4.1 Synthèse des droits

Ressource	PUBLIC	CUSTOMER	PARTNER	ADMIN
Categories	Lire	Lire	Lire	Gérer
Partners	Lire	Lire	Lire	Gérer
Products	Lire	Lire	Gérer ses produits	Gérer tout
Customers	Non	Son profil	Clients ayant commandé ses produits	Tous
Orders	Non	Ses commandes	Commandes contenant ses produits	Toutes
Payments	Non	Paiements de ses commandes	Paiements liés à ses produits	Tous
Deliveries	Non	Ses livraisons	Livraisons liées à ses produits	Toutes

5. Modèle de données mis à jour

5.1 Ajouts dans users

```
users
- id
- name
- email
- password
- role // ADMIN, PARTNER, CUSTOMER
```

5.2 Liaison customer vers user

```
customers.id_users -> users.id
Objectif : relier un profil client à un compte authentifié.
```

5.3 Table partner_users

```
partner_users
- id
- id_users
- id_partners
- state
- created_at
- updated_at
Objectif : relier un utilisateur PARTNER à un ou plusieurs restaurants.
```

5.4 Ajout dans payment

```
payment.phone_payment
Objectif : stocker le numéro utilisé pour le paiement Mobile Money ou le paiement à la livraison.
```

6. Authentification API

Les routes sensibles nécessitent un token transmis dans le header Authorization.

Authorization: Bearer TOKEN

6.1 Inscription client

```
POST /api/auth/register
{
  "first_name_customers": "Franck",
  "last_name_customers": "Kianguebeni",
  "phone_customers": "066000001",
  "mail_customers": "franck2@test.com",
  "password": "password123",
  "password_confirmation": "password123"
}
```

6.2 Connexion

```
POST /api/auth/login
{
  "mail_customers": "partner@godfavor.com",
  "password": "password123"
}
```

La réponse retourne le token, le type Bearer, l'utilisateur et son rôle.

6.3 Utilisateur connecté

```
GET /api/auth/me
Header: Authorization: Bearer TOKEN
```

6.4 Déconnexion

```
POST /api/auth/logout
Header: Authorization: Bearer TOKEN
```

7. Routes publiques

Ces routes sont accessibles sans token. Elles servent au catalogue et à l'authentification initiale.

```
GET /api/health
POST /api/auth/register
POST /api/auth/login
GET /api/categories
GET /api/categories/{id}
GET /api/partners
GET /api/partners/{id}
GET /api/products
GET /api/products/{id}
```

8. Routes protégées

Ces routes nécessitent un token Sanctum. Les permissions précises dépendent du rôle utilisateur.

```
GET /api/auth/me
POST /api/auth/logout
GET /api/my-products
POST /api/products
PUT /api/products/{id}
DELETE /api/products/{id}
GET /api/customers
GET /api/customers/{id}
PUT /api/customers/{id}
DELETE /api/customers/{id}
GET /api/orders
POST /api/orders
GET /api/orders/{id}
PUT /api/orders/{id}
DELETE /api/orders/{id}
GET /api/payments
POST /api/payments
GET /api/payments/{id}
PUT /api/payments/{id}
DELETE /api/payments/{id}
GET /api/deliveries
POST /api/deliveries
GET /api/deliveries/{id}
PUT /api/deliveries/{id}
DELETE /api/deliveries/{id}
```

9. Sécurité des produits

Un utilisateur PARTNER lié à God Favor peut créer un produit uniquement pour ce partenaire. Toute tentative pour un autre partenaire existant est refusée.

9.1 Cas autorisé

```
POST /api/products
Authorization: Bearer TOKEN_PARTNER
{
  "label_products": "Burger God Favor",
  "price": 7000,
  "id_category": 2,
  "id_partners": 1
}
Résultat : 201 Created
```

9.2 Cas interdit

```
POST /api/products
Authorization: Bearer TOKEN_PARTNER
{
  "label_products": "Produit interdit",
  "price": 5000,
  "id_category": 1,
  "id_partners": 2
}
Résultat : 403 Forbidden
{
  "success": false,
  "message": "Accès refusé. Vous ne pouvez créer un produit que pour votre propre partenaire."
}
```

10. Sécurité des commandes

Rôle	Règle appliquée
ADMIN	Voit toutes les commandes et peut modifier les statuts globaux.
PARTNER	Voit uniquement les commandes contenant au moins un produit de son partenaire. Peut mettre à jour certains statuts opérationnels.
CUSTOMER	Crée ses commandes avec son token et voit uniquement ses commandes.

10.1 Création de commande par le client connecté

Le client ne doit plus envoyer id_customers. Le client est identifié automatiquement à partir du token.

```
POST /api/orders
Authorization: Bearer TOKEN_CUSTOMER
{
  "products": [
    {
      "id_products": 1,
      "quantity": 2
    }
  ]
}
```

10.2 Visibilité partenaire

```
orders
-> order_details
-> products
-> partners
```

Le PARTNER ne voit que les commandes ayant des produits liés à ses id_partners.

11. Sécurité des clients

Rôle	Règle appliquée
ADMIN	Voit et gère tous les clients.
PARTNER	Voit uniquement les clients ayant commandé ses produits.
CUSTOMER	Voit et modifie uniquement son propre profil.

12. Paiements

Le module payment prend en charge le paiement classique, le paiement Mobile Money et le paiement à la livraison. Le champ `phone_payment` permet de tracer le numéro utilisé.

12.1 Exemple paiement Mobile Money

```
POST /api/payments
Authorization: Bearer TOKEN_CUSTOMER
{
  "id_orders": 1,
  "type": "MOBILE_MONEY",
  "phone_payment": "0660000000",
  "transaction": "TXN-CUST-001",
  "token": "TOKEN-CUST-001",
  "response": "OK",
  "status_transaction": "SUCCESS"
}
```

12.2 Exemple paiement à la livraison

```
POST /api/payments
Authorization: Bearer TOKEN_CUSTOMER
{
  "id_orders": 1,
  "type": "CASH_ON_DELIVERY",
  "phone_payment": "0660000000",
  "transaction": "CASH-001",
  "response": "OK",
  "status_transaction": "SUCCESS"
}
```

12.3 Règles de sécurité paiement

Rôle	Règle appliquée
ADMIN	Voit et gère tous les paiements.
CUSTOMER	Peut payer uniquement ses propres commandes.
PARTNER	Voit les paiements liés à ses produits, mais ne peut pas créer un paiement.

Test validé

```
POST /api/payments avec token PARTNER
Résultat : 403 Forbidden
{
  "success": false,
  "message": "Accès refusé. Un partenaire ne peut pas créer un paiement."
}
```

13. Référentiel officiel des statuts métier

Référence fonctionnelle officielle : ces valeurs doivent être utilisées par le backend, le back-office, les applications mobiles, les partenaires et les intégrations API.

13.1 Statuts de commande - status_order

Statut	Description
PENDING	Commande créée, en attente de validation.
CONFIRMED	Commande acceptée par le partenaire.
PREPARING	Commande en cours de préparation.
READY	Commande prête à être remise au livreur.
COMPLETED	Commande totalement terminée.
CANCELLED	Commande annulée.
REJECTED	Commande refusée par le partenaire.

Workflow recommandé commande :

PENDING -> CONFIRMED -> PREPARING -> READY -> COMPLETED

Cas alternatifs :

PENDING -> CANCELLED

PENDING -> REJECTED

13.2 Statuts de paiement - status_payment

Statut	Description
PENDING	Paieement non effectué.
PROCESSING	Paieement en cours de traitement.
PAID	Paieement validé.
FAILED	Paieement échoué.
REFUNDED	Paieement remboursé.
CANCELLED	Paieement annulé.

Paieement avant livraison :

PENDING -> PROCESSING -> PAID

PENDING -> PROCESSING -> FAILED

Paieement à la livraison :

Création commande : status_payment = PENDING

Après encaissement : status_payment = PAID

13.3 Statuts de livraison - status_delivery

Statut	Description
PENDING	Livraison créée.
ASSIGNED	Livreur affecté.
PICKED_UP	Commande récupérée.
ON_THE_WAY	Livraison en cours.
DELIVERED	Livraison effectuée.
FAILED	Échec de livraison.
CANCELLED	Livraison annulée.

Workflow recommandé livraison :

PENDING -> ASSIGNED -> PICKED_UP -> ON_THE_WAY -> DELIVERED

Cas alternatifs :

PENDING -> FAILED

PENDING -> CANCELLED

13.4 Exemple complet de cycle métier

Commande créée :
 status_order = PENDING
 status_payment = PENDING
 status_delivery = PENDING

Restaurant accepte : status_order = CONFIRMED
 Préparation : status_order = PREPARING
 Commande prête : status_order = READY
 Paiement validé : status_payment = PAID
 Livreur affecté : status_delivery = ASSIGNED
 Commande récupérée : status_delivery = PICKED_UP
 Livraison en cours : status_delivery = ON_THE_WAY
 Livraison effectuée: status_delivery = DELIVERED
 Fin processus : status_order = COMPLETED

13.5 Valeurs autorisées dans l'API

Champ	Valeurs autorisées
status_order	PENDING, CONFIRMED, PREPARING, READY, COMPLETED, CANCELLED, REJECTED
status_payment	PENDING, PROCESSING, PAID, FAILED, REFUNDED, CANCELLED
status_delivery	PENDING, ASSIGNED, PICKED_UP, ON_THE_WAY, DELIVERED, FAILED, CANCELLED

14. Livraisons

14.1 Création de livraison

```
POST /api/deliveries
Authorization: Bearer TOKEN_ADMIN
{
  "id_orders": 1,
  "area_delivery": "Centre-ville Pointe-Noire",
  "status": "PENDING"
}
```

14.2 Statuts recommandés

PENDING
 ASSIGNED
 PICKED_UP
 ON_THE_WAY
 DELIVERED
 FAILED
 CANCELLED

14.3 Droits livraison

Rôle	Règle appliquée
ADMIN	Voit toutes les livraisons, peut créer, modifier et supprimer.
PARTNER	Voit les livraisons liées à ses produits et peut mettre à jour le statut si concerné.
CUSTOMER	Voit uniquement ses livraisons et ne peut pas modifier le statut.

15. Déploiement et configuration serveur

15.1 Configuration production .env

```
APP_NAME="Miam N Go"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://api-miamngo.digiapp.tech
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=c2723812c_miam_n_go
DB_USERNAME=c2723812c_miam_n_go
DB_PASSWORD=*****
```

15.2 Commandes serveur

```
cd ~/public_html/api-miamngo
php artisan migrate --force
php artisan optimize:clear
php artisan route:clear
composer dump-autoload
```

15.3 Commandes Git

```
git add .
git commit -m "Ajout authentification Sanctum et securite par roles"
git push origin main
```

16. État actuel du projet

Module	État
Production HTTPS	Validé
GitHub	Synchronisé
MySQL production	Opérationnel
Category / Partner / Product	Opérationnels
Customer / Order / OrderDetail	Opérationnels et sécurisés
Payment	Opérationnel avec phone_payment et paiement à la livraison
Delivery	Opérationnel avec statuts de livraison
Auth Sanctum	Opérationnel
Rôles ADMIN / PARTNER / CUSTOMER	Intégrés
Référentiel des statuts métier	Ajouté en v1.3
Sécurité produits	Validée
Sécurité paiements	Validée
Sécurité commandes, clients, livraisons	Intégrée et à valider complètement par tests de non-régression

17. Tests validés

```

POST /api/auth/register      OK
POST /api/auth/login        OK
GET  /api/auth/me           OK
POST /api/products          OK avec contrôle partenaire
GET  /api/my-products       OK
POST /api/orders            OK
POST /api/payments          OK avec CUSTOMER
POST /api/payments          Refusé avec PARTNER - 403
POST /api/deliveries        OK
PARTNER produit autre partenaire Refusé - 403

```

18. Prochaines étapes

Priorité	Action
1	Finaliser les tests DeliveryController avec ADMIN, PARTNER et CUSTOMER.
2	Effectuer un commit Git et un déploiement production.
3	Tester les routes de production avec tokens.
4	Préparer une collection Postman officielle.
5	Ajouter un vrai module de panier carts et cart_details.
6	Préparer l'intégration mobile Flutter ou web app.

19. Conclusion

Miam N Go dispose maintenant d'une API REST Laravel solide, déployée en production, authentifiée par tokens, structurée pour gérer les droits des clients, partenaires et administrateurs, et dotée d'un référentiel officiel des statuts métier.

L'API constitue une base professionnelle pour une application web et mobile de commande et livraison de repas.

URL officielle : <https://api-miamngo.digiapp.tech>